

Lecture 5

Model Selection

5.1

Contents

1	Introduction	1
2	Subset Selection	4
2.1	Best Subset Selection	5
2.1.1	How does Best Subset Selection Solve Multicollinearity?	5
2.2	Stepwise Selection Methods	6
2.2.1	Forward Stepwise Selection	6
2.2.2	How does Forward Search Selection Solve Multicollinearity?	7
2.2.3	Backward Stepwise Selection	8
2.2.4	Hybrid Approaches	9
2.3	Choosing the Optimal Model	9
2.4	Example	11

Some figures in this presentation are taken from 'An Introduction to Statistical Learning, with applications in R' (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani.

Most of the material is extracted from the slides of the MOOC course by Hastie and Tibshirani, available at <https://www.r-bloggers.com/in-depth-introduction-to-machine-learning-in-15-hours-of-expert-videos/>.

5.2

1 Introduction

The linear model

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon$$

has good inferential properties, and its coefficients β_i can be easily interpreted. On real-world problems is often surprisingly competitive in relation to non-linear methods.

5.3

Usually, linear model parameters are estimated by Ordinary Least Squares

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{n} \sum_{i=1}^n [y_i - (\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})]^2 \right\}.$$

We will consider methods alternative to OLS for estimating a linear model, that can improve the predictive ability and model interpretability, and can give solutions to the multicollinearity problem.

5.4

Predictive ability

- Provided that the true relationship f between the response Y and the predictors $X_1, \dots, X_p$ is approximately linear, the least squares estimate $\hat{f}$ will have low bias.
- If $n \gg p$ OLS parameter estimates will have low standard errors, therefore, the least squares estimate $\hat{f}$ tend to also have low variance, and hence will perform well on test observations.
- If n is not much larger than p then there can be a lot of variability in the least squares fit, resulting in overfitting, and consequently poor predictions on future observations not used in model training.

5.5

- If $p > n$ the model is no more identifiable, and there is no longer a unique least squares coefficient estimate. The variance of the 'linear model' method is infinite so the method cannot be used at all.

By constraining or shrinking the estimated coefficients, we can often substantially reduce the variance at the cost of a negligible increase in bias.

This can lead to substantial improvements in the accuracy with which we can predict the response for observations not used in model training.

- If $n = p + 1$ and the model matrix (made up by the unitary array and by the arrays containing values of the p predictors) has rank $p + 1$, the model is identified, but it is not possible to obtain an estimate of the error variance. The response variable vector belongs to the linear subspace defined by the columns of the model matrix of regressors and residuals will be null.

5.6

Example 1 (Simulation of the parameter estimates in a model with 2 regressors and $n = 3$ observations). We consider the following fixed values for the regressors

```
x1 <- c(-2, 1, 1)
x2 <- c(0, -1, 1)
```

Regressors are uncorrelated as shown by their correlation matrix

```
cor(x1, x2)
## [1] 0
```

The following function generates an outcome y according to a linear relationship and estimates the corresponding linear model, returning coefficients estimates.

```
f <- function() {
  y <- 30 + 5 * x1 + 10 * x2 + rnorm(3)
  c(coef(lm(y ~ x1 + x2)))
}
```

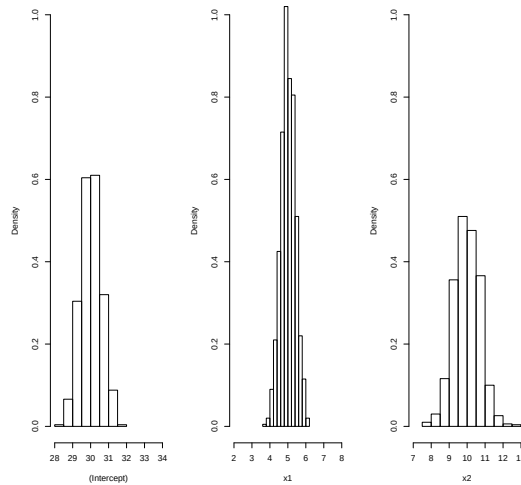
5.7

We consider a simulation with 1000 replicates

```
set.seed(123)
a <- replicate(1000, f())
```

and represent coefficients estimates in the 1000 replicates with histograms

```
layout(matrix(1:3, 1, 3))
xlim_list <- list(c(28, 34), c(2, 8), c(7, 13))
sapply(1:3, function(i) {
  hist(a[i, ], main = "", xlab = rownames(a)[i], freq = FALSE,
       xlim = xlim_list[[i]], ylim = 0:1)
})
```



Remembering that σ_E^2 was fixed to 1, we can find the theoretical variance of the OLS estimator

```
X <- cbind(1, x1, x2)
diag(solve(t(X) %*% X))
##              x1              x2
## 0.33333333 0.16666667 0.50000000
```

and empirical values of the variance of the estimates are close to theoretical

```
apply(a, 1, var)
## (Intercept)              x1              x2
## 0.3255358    0.1605899    0.5103282
```

We can observe that in this situation residuals, corresponding to each estimate of the model, are null

```
y <- 30 + 5 * x1 + 10 * x2 + rnorm(3, sd = 1)
residuals(out <- lm(y ~ x1 + x2))
## 1 2 3
## 0 0 0
```

and the variance of the error cannot be estimated

$$\hat{\sigma}^2 = \frac{1}{n-p-1} \sum_{i=1}^n \hat{\epsilon}_i^2 = \frac{1}{3-2-1} \cdot 0 = \frac{1}{0} \cdot 0$$

since we have $n-p-1 = 3-2-1 = 0$.

Therefore, no inference result can be produced

```
summary(out)
##
## Call:
## lm(formula = y ~ x1 + x2)
##
## Residuals:
## ALL 3 residuals are 0: no residual degrees of freedom!
##
## Coefficients:
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)    29.358         NA      NA      NA
## x1             4.754         NA      NA      NA
## x2             9.440         NA      NA      NA
##
## Residual standard error: NaN on 0 degrees of freedom
## Multiple R-squared:      1, Adjusted R-squared:      NaN
## F-statistic:      NaN on 2 and 0 DF,  p-value: NA
```

5.12

In this situation we have $MSE_{training} = 0$.

The estimation method is characterized by overfitting since it exactly fits the observations, but fails in fitting the model. In presence of new observations we will find $MSE_{test} > 0$.

We have similar situations of overfitting whenever $n \gg p$; in these situations $MSE_{training} \simeq 0$ and $MSE_{test} \gg MSE_{training}$.

5.13

Model interpretability

It is often the case that some or many of the variables used in a multiple regression model with $p \gg 1$ are in fact not associated with the response.

Including such irrelevant variables leads to unnecessary complexity in the resulting model. By removing these variables – that is, by setting the corresponding coefficient estimates to zero – we can obtain a model that is more easily interpreted. Now least squares is extremely unlikely to yield any coefficient estimates that are exactly zero.

We consider some approaches for automatically performing feature selection or variable selection – that is, for excluding irrelevant variables from a multiple regression model.

5.14

Alternative methods to fit a multiple linear regression model

- *Subset selection.*

This approach involves identifying a subset of the p predictors that we believe to be related to the response. We then fit a model using least squares on the reduced set of variables.

- *Shrinkage.*

This approach involves fitting a model involving all p predictors.

However, the estimated coefficients are shrunk towards zero relative to the least squares estimates. This shrinkage (also known as regularization) has the effect of reducing variance.

Depending on what type of shrinkage is performed, some of the coefficients may be estimated to be exactly zero (Lasso). Hence, shrinkage methods can also perform variable selection.

- *Dimension Reduction.*

This approach involves projecting the p predictors into a M -dimensional subspace, where $M < p$. This is achieved by computing M different linear combinations, or projections, of the variables. Then these M projections are used as predictors to fit a linear regression model by least squares.

5.15

2 Subset Selection

We consider some methods for selecting subsets of predictors with reference to the model

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon.$$

Observe that the same concepts apply also to other methods, such as generalized linear models or also to classification methods.

5.16

2.1 Best Subset Selection

First, all the 2^p models, that can be constructed with predictors $X_1, \dots, X_p$, are fitted with OLS. Then, the *best* model is chosen.

By using a subset of the p predictors all the models resumed in the following table can be constructed

n. predictors	$0 \equiv \text{null model}$	1	2	...	$p-1$	p
n. of possible models	$1 = \binom{p}{0}$	$p = \binom{p}{1}$	$\binom{p}{2}$	...	$p = \binom{p}{p-1}$	$1 = \binom{p}{p}$

Selection is made within 2^p models. Namely, we have:

$$\sum_{i=0}^p \binom{p}{i} = \sum_{i=0}^p \binom{p}{i} 1^i 1^{p-i} = (1+1)^p = 2^p.$$

5.17

Best subset selection algorithm

1. Let $\mathcal{M}_0$ denote the null model $Y = \beta_0 + \varepsilon$, which contains no predictors. This model simply predicts the sample mean for each observation $\hat{\beta}_0 = \bar{y}$.
2. for (k in $1:p$)
 - (a) Fit all $\binom{p}{k}$ models that contain exactly k predictors.
 - (b) Pick the best among these $\binom{p}{k}$ models, and call it $\mathcal{M}_k$. Here *best* is defined as having the smallest $MSE_{training}$, or equivalently largest R^2 .
3. Select a single best model from among $\{\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p\}$ using *crossvalidated* prediction error (an estimate of MSE_{test}), or *mathematical adjustments* of $MSE_{training}$ (C_p , AIC , BIC or adjusted R^2).

5.18

A disadvantage of best subset selection

The procedure is not feasible in presence of a high number of variables ($p \gg 1$). The complexity level increases exponentially:

- if $p = 5$ we have to estimate 32 models,
- if $p = 10$ we have to estimate 1024 models,
- if $p = 15$ we have to estimate 3.2768×10^4 models,
- if $p = 20$ we have to estimate 1.048576×10^6 models.

5.19

2.1.1 How does Best Subset Selection Solve Multicollinearity?

Let assume that only the two variables X_v and X_w , belonging to the set of the possible predictors $\mathcal{X}_p = \{X_1, \dots, X_p\}$ of the outcome Y , are affected by multicollinearity issues; that is $|\rho_{X_v, X_w}| > 0.95$ (being $n \gg 1$).

5.20

The Best Subset selection method considers, in the different steps of the algorithm, models containing only X_v , only X_w , or both predictors.

Let us restrict the analysis to models that contain the same set of the remaining variables:

- all these models will have close fits;
- the Best Subset selection algorithm will choose, at step 2, the model characterized by minimum $MSE_{training}$, among models containing only X_v or X_w .
Let call $\mathcal{M}_o$ this model and $\mathcal{M}_b$ the model with both variables X_v and X_w ; we have $b > o$.
- We expect, according to considerations made at step 3. of the algorithm, or, anyway, for parsimony, that model $\mathcal{M}_o$ is preferred to model $\mathcal{M}_b$.

5.21

2.2 Stepwise Selection Methods

Stepwise methods, which explore a far more restricted set of models, are attractive alternatives to best subset selection.

5.22

2.2.1 Forward Stepwise Selection

Forward stepwise selection begins with a model containing no predictors, and then adds predictors to the model, one-at-a-time, until all of the predictors are in the model. At each step the variable that gives the greatest *additional* improvement to the fit is added to the model.

5.23

Forward Stepwise Selection algorithm

1. Let $\mathcal{M}_0$ denote the null model $Y = \beta_0 + \varepsilon$, which contains no predictors. This model simply predicts the sample mean for each observation $\hat{\beta}_0 = \bar{y}$.
2. for (k in $0 : (p - 1)$)
 - (a) Fit all $p - k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
 - (b) Choose the best among these $p - k$ models, and call it $\mathcal{M}_{k+1}$. Here best is defined as having smallest $MSE_{training}$ or highest R^2 .
3. Select a single best model from among $\{\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p\}$ using *crossvalidated* prediction error (an estimate of MSE_{test}), or *mathematical adjustments* of $MSE_{training}$ (C_p , AIC , BIC or adjusted R^2).

5.24

Advantage

Unlike best subset selection, which involved fitting 2^p models, forward stepwise selection involves fitting one null model, along with $p - k$ models in the k th iteration, for $k = 0, \dots, p - 1$. This amounts to a total of

$$1 + \sum_{k=0}^{p-1} (p - k) = 1 + \sum_{j=1}^p j = 1 + \frac{p(p+1)}{2}$$

models. This is a substantial difference

5.25

- if $p = 5$, forward stepwise selection requires fitting 15 models whereas best subset selection requires 32 models,
- if $p = 10$, forward stepwise selection requires fitting 55 models whereas best subset selection requires 1024 models,
- if $p = 15$, forward stepwise selection requires fitting 120 models whereas best subset selection requires 3.2768×10^4 models,
- if $p = 20$, forward stepwise selection requires fitting 210 models whereas best subset selection requires 1.048576×10^6 models.

Though forward stepwise selection considers $\frac{p(p+1)}{2} + 1$ models, it performs a *guided* search over model space, and so the effective model space considered contains substantially more than $\frac{p(p+1)}{2} + 1$ models.

5.26

Drawback

It is not guaranteed to find the best possible model out of all 2^p models containing subsets of the p predictors.

5.27

Example 2. Assume $p = 3$ and denote with X_1, X_2, X_3 the predictors.

If with best subset selection we have

- best model $\mathcal{M}_1 = \hat{f}(X_1)$
- best model $\mathcal{M}_2 = \hat{f}(X_2, X_3)$

the search of the best model by means of Forward Stepwise Selection will result in

- best model $\mathcal{M}_1 = \hat{f}(X_1)$
- best model $\mathcal{M}_2 = \hat{f}(X_1, X_i), i \in \{2, 3\}$

The final selection with Forward Stepwise Selection will be in one of the 2 sets

$$\{\mathcal{M}_1 = \hat{f}(X_1), \mathcal{M}_2 = \hat{f}(X_1, X_2)\} \quad \text{or} \quad \{\mathcal{M}_1 = \hat{f}(X_1), \mathcal{M}_2 = \hat{f}(X_1, X_3)\}$$

and not in the set

$$\{\mathcal{M}_1 = \hat{f}(X_1), \mathcal{M}_2 = \hat{f}(X_2, X_3)\}$$

provided by best subset selection.

5.28

Remark

The Forward Stepwise Selection applies also if $n < p$ (high dimensional setting). However, in this case, it is possible to construct sub-models

$$\{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_{n-1}\}$$

only, since each sub-model is fit using least squares, which will not yield a unique solution if $p \geq n$.

5.29

2.2.2 How does Forward Search Selection Solve Multicollinearity?

The Forward Stepwise selection algorithm, can be re-written in an equivalent way that gives evidence of its ability in dealing with multicollinearity.

First we give the definition of partial correlation coefficient within a regression model.

5.30

Definition 3 (The Partial Correlation Coefficient). Let $X_1, \dots, X_p$ be the candidate predictors in a linear regression model, *complete* of intercept, that is used to describe a response variable Y .

Let:

- $\mathcal{X}_k \subset \mathcal{X}_p = \{X_1, \dots, X_p\}$ a subset of the p predictors
- $Y^* = Y - f(\mathcal{X}_k)$ the residuals of the linear regression model, complete of intercept, that explains Y as a function of the predictors in $\mathcal{X}_k$.
- $X_j^* = X_j - f(\mathcal{X}_k)$ the residuals of the linear regression model, complete of intercept, that explains $X_j \notin \mathcal{X}_k$ as a function of the predictors in $\mathcal{X}_k$.

The partial correlation coefficient between Y and X_j , given the predictors in $\mathcal{X}_k$, is denoted with $\rho_{YX_j \cdot \mathcal{X}_k}$, and is equal to the linear correlation coefficient between Y^* and X_j^*

$$\rho_{YX_j \cdot \mathcal{X}_k} = \rho_{Y^*X_j^*}$$

5.31

Step 2 of the Forward Stepwise selection algorithm can be replaced with

1. for (k in $0 : (p - 1)$)
 - (a) Compute the partial correlation coefficients between Y and each new predictor (chosen within the set of variables belonging to $\mathcal{X}_p$ but not to $\mathcal{X}_k$), given the variables already present in the model (in $\mathcal{X}_k$).
 - (b) Call $\mathcal{M}_{k+1}$ the model obtained by adding to $\mathcal{M}_k$ the predictor with maximum absolute partial correlation with Y , given the variables already present in the model, as result from the preceding step.

Assume the the two variables $(X_v, X_w) \in \mathcal{X}_p$ present multicollinearity issues, that is $VIF(X_v) = \frac{1}{1-R_v^2} > 10$ and $VIF(X_w) = \frac{1}{1-R_w^2} > 10$ since $|\rho_{X_v X_w}| > 0.95$ (being $n \gg 1$). We also assume for simplicity that $|\rho_{Y X_w}| > |\rho_{Y X_v}|$.

The variable in $\mathcal{X}_p$ that has highest correlation with Y is included in the model at step $k = 0$ of the Forward stepwise selection algorithm. Let assume, for simplicity, that it is X_w .

Step $k = 1$ will choose within the set $X_j, j \in \{1, 2, \dots, p\}, j \neq w$, the variable with maximum absolute partial correlation with Y

$$|\rho_{Y X_j \cdot X_w}|,$$

that is the variable giving the maximum marginal contribution in explaining the variability of Y , given the contribution of X_w , already present in the model.

Usually at least one of the predictors $X_j, j \in \{1, 2, \dots, p\}, j \notin \{w, v\}$, will have

$$|\rho_{Y X_j \cdot X_w}| > |\rho_{Y X_v \cdot X_w}|$$

that is a marginal contribution to the variability Y higher than X_v .

Variable X_v that presented multicollinearity issues with X_w will be inserted in the model in one of the successive iterations of the Forward Stepwise selection algorithm, that ends with inserting all variables in the model.

2.2.3 Backward Stepwise Selection

Like forward stepwise selection, backward stepwise selection provides an efficient alternative to best subset selection. However, unlike forward stepwise selection, it begins with the full least squares model containing all p predictors $X_1, X_2, \dots, X_p$, and then iteratively removes the least useful predictor, one-at-a-time.

At each iteration it removes the variable that, conditional on all other variables, gives the minimum contribution to explain the variability of the response.

That variable has the minimum reduction of the fit measure R^2 , and correspondingly the minimum increase of $MSE_{training}$. The procedure stops when all variables are excluded from the model.

Backward Stepwise Selection Algorithm

1. Let $\mathcal{M}_p$ denote the full model, which contains all p predictors.

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon.$$

2. for (k in $p : 1$)
 - (a) Consider all k models that contain all but one of the predictors in $\mathcal{M}_k$, for a total of $k - 1$ predictors.
 - (b) Choose the best among these k models, and call it $\mathcal{M}_{k-1}$. Here best is defined as having smallest RSS or highest R^2 .
3. Select a single best model from among $\{\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p\}$ using *crossvalidated* prediction error (an estimate of MSE_{test}), or *mathematical adjustments of the $MSE_{training}$* (C_p , AIC , BIC o adjusted R^2).

Advantage

Like forward stepwise selection, the backward selection approach searches through only

$$1 + \sum_{k=0}^{p-1} (p-k) = 1 + \sum_{j=1}^p j = 1 + \frac{p(p+1)}{2}$$

models, and so can be applied in settings where p is too large to apply best subset selection.

Like forward stepwise selection, backward stepwise selection performs a guided search over model space, and so effectively considers substantially more than $1 + \frac{p(p+1)}{2}$ models.

5.38

Drawback

Also like forward stepwise selection, backward stepwise selection is not guaranteed to yield the best model containing a subset of the p predictors.

The Backward Stepwise Selection method can be applied only if $n > p$.

5.39

2.2.4 Hybrid Approaches

As another alternative, hybrid versions of forward and backward stepwise selection are available, in which variables are added to the model sequentially, in analogy to forward selection. However, after adding each new variable, the method may also remove any variables that no longer provide an improvement in the model fit.

Some softwares define which variables have to be included in the model or excluded from the model according to input and output thresholds related to an F statistic (or equivalently to the t statistic, since $F = t^2$) related to the significance of each predictor in explaining the response Y .

5.40

Remarks

Such an approach attempts to more closely mimic best subset selection while retaining the computational advantages of forward and backward stepwise selection.

Pay always high care when using criteria based only on inferential rules. If the decision of dropping a variable is based on its non-significance (e.g. by looking at the fact that its p -value is larger than 0.05), one has to take into account that in presence of a high number of variables, a proportion of 5% of the variables could result significant only by chance.

5.41

2.3 Choosing the Optimal Model

At step 2 the preceding algorithms select the model with minimum $MSE_{training}$.

At step 3 the selection is made basing on the estimate of the MSE_{test} ;

we remember that, with regard to prediction, the $MSE_{training}$ underestimates the MSE_{test} . Therefore $MSE_{training}$ and R^2 are not suitable for selecting the best model in the set $\{\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p\}$.

For example, with reference to the nested models

$$\{\mathcal{M}_0 \subset \mathcal{M}_1 \subset \dots \subset \mathcal{M}_p\}$$

$MSE_{training}$ is a decreasing function of the number of variables in the model and cannot be used for variable/model selection.

5.42

Solutions

- estimate of MSE_{test} by means of cross validation, see next section.
- indirect estimate of MSE_{test} by the following mathematical adjustments of $MSE_{training}$
 - Mallows's C_p
 - AIC
 - BIC
 - adjusted R^2

We define the training Residual Sum of Squares as

$$RSS_{training} = \sum_{i=1}^n (y_i - \hat{f}(x_{i1}, \dots, x_{ip}))^2$$

It holds

$$MSE_{training} = \frac{1}{n} RSS_{training}$$

Mallow's C_p

For a linear model fitted with OLS, Mallow's C_p is defined as

$$C_p = \frac{1}{n} (RSS_{training} + 2d\hat{\sigma}^2)$$

where

- d is the number of predictors in the linear regression model
- $\hat{\sigma}^2$ is the estimate of σ_ε^2 , variance of the error in the full model containing all p predictors

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon.$$

The penalty $2d\hat{\sigma}^2$ increases with the number d of predictors.

The Mallow's C_p statistic is sometimes defined as

$$C'_p = \frac{RSS_{training}}{\hat{\sigma}^2} + 2d - n.$$

The two statistics are equivalent with reference to the selection problem; namely, we have

$$C_p = \frac{1}{n} \hat{\sigma}^2 (C'_p + n).$$

Akaike Information Criterion (AIC)

The AIC criterion is used with models estimated with maximum likelihood; in presence of a linear model we have

$$AIC = \frac{1}{n\hat{\sigma}^2} (RSS_{training} + 2d\hat{\sigma}^2)$$

where

- d is the number of predictors
- $\hat{\sigma}^2$ is the estimate of σ_ε^2 , variance of the error component in the model with all the p predictors.

AIC is proportional to the Mallow's C_p statistic.

Schwarz Bayesian Information Criterion (BIC)

The BIC criterion comes from a Bayesian approach, and is used, like the AIC, for models estimated with maximum likelihood. In presence of a linear model it is defined as

$$BIC = \frac{1}{n} (RSS_{training} + \ln(n)d\hat{\sigma}^2)$$

where

- d is the number of predictors
- $\hat{\sigma}^2$ is the estimate of σ_{ε}^2 , variance of the error component in the model with all the p predictors.

The penalty $\ln(n)d\hat{\sigma}^2$ term increases with both the sample size as well as the number of predictors.

5.48

Adjusted R^2

$$\text{Adjusted } R^2 = 1 - \frac{RSS_{\text{training}}/(n-d-1)}{TSS/(n-1)}$$

where

- d is the number of predictors
- $TSS = \sum_{i=1}^n (y_i - \bar{y})^2 = n\hat{\sigma}_Y^2$ (Total sum of squares) is the total of squared deviations from the sample mean.

See lectures on the linear model.

5.49

2.4 Example

We reproduce results by JWHT on the Credit data set, that the authors used as illustration in the presentation of subset selection methods.

Data are imported from the `Credit.csv` data set and its content checked.

```
credit <- read.table("Credit.csv", sep = ",", header = T)
head(credit)
##      X  Income Limit Rating Cards Age Education Gender Student
## 1 1  14.891  3606   283     2  34         11  Male      No
## 2 2 106.025  6645   483     3  82         15 Female     Yes
## 3 3 104.593  7075   514     4  71         11  Male      No
## 4 4 148.924  9504   681     3  36         11 Female     No
## 5 5  55.882  4897   357     2  68         16  Male      No
## 6 6  80.180  8047   569     4  77         10  Male      No
##      Married Ethnicity Balance
## 1      Yes Caucasian    333
## 2      Yes   Asian    903
## 3       No   Asian    580
## 4       No   Asian    964
## 5      Yes Caucasian    331
## 6       No Caucasian   1151
```

5.50

Since the first variable contains the subject codes, we drop it.

```
credit <- credit[, -1]
```

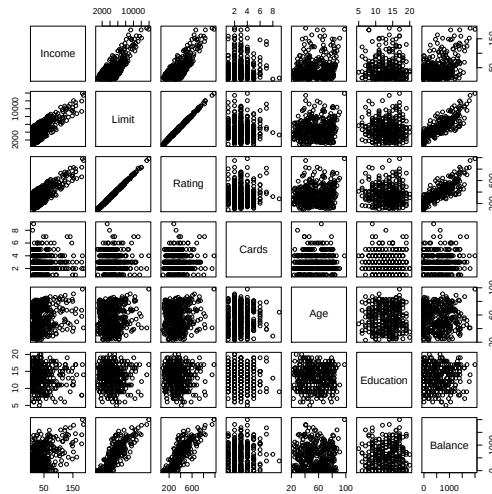
We check if not available cases are present and find the number of subjects.

```
max(is.na(credit))
## [1] 0
(n <- nrow(credit))
## [1] 400
```

5.51

A scatter plot matrix (excluding categorical variables) is produced,

```
plot(credit[, -(7:10)])
```



5.52

and the correlation matrix (across metric variables) obtained

```
cor(credit[, -(7:10)])
```

	Income	Limit	Rating	Cards
Income	1.00000000	0.79208834	0.79137763	-0.01827261
Limit	0.79208834	1.00000000	0.99687974	0.01023133
Rating	0.79137763	0.99687974	1.00000000	0.05323903
Cards	-0.01827261	0.01023133	0.05323903	1.00000000
Age	0.17533840	0.10088792	0.10316500	0.04294829
Education	-0.02769198	-0.02354853	-0.03013563	-0.05108422
Balance	0.46365646	0.86169727	0.86362516	0.08645635

	Age	Education	Balance
Income	0.175338403	-0.027691982	0.463656457
Limit	0.100887922	-0.023548534	0.861697267
Rating	0.103164996	-0.030135627	0.863625161
Cards	0.042948288	-0.051084217	0.086456347
Age	1.000000000	0.003619285	0.001835119
Education	0.003619285	1.000000000	-0.008061576
Balance	0.001835119	-0.008061576	1.000000000

5.53

The variables Limit and Rating are highly correlated.

A linear model explaining Balance as a function of all other variables is considered.

```
output <- lm(Balance ~ ., data = credit)
summary(output)
```

```
##
## Call:
## lm(formula = Balance ~ ., data = credit)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -161.64  -77.70  -13.49   53.98  318.20
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -479.20787     35.77394  -13.395  < 2e-16 ***
```

```
## Income -7.80310 0.23423 -33.314 < 2e-16 ***
## Limit 0.19091 0.03278 5.824 1.21e-08 ***
## Rating 1.13653 0.49089 2.315 0.0211 *
## Cards 17.72448 4.34103 4.083 5.40e-05 ***
## Age -0.61391 0.29399 -2.088 0.0374 *
## Education -1.09886 1.59795 -0.688 0.4921
## GenderFemale -10.65325 9.91400 -1.075 0.2832
## StudentYes 425.74736 16.72258 25.459 < 2e-16 ***
## MarriedYes -8.53390 10.36287 -0.824 0.4107
## EthnicityAsian 16.80418 14.11906 1.190 0.2347
## EthnicityCaucasian 10.10703 12.20992 0.828 0.4083
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 98.79 on 388 degrees of freedom
## Multiple R-squared: 0.9551, Adjusted R-squared: 0.9538
## F-statistic: 750.3 on 11 and 388 DF, p-value: < 2.2e-16
```

5.54

The Variance Inflation Factor confirms that the variables `Limit` and `Rating` are affected by multicollinearity issues.

```
library(car)
## Loading required package: carData
vif(output)
##              GVIF Df GVIF^(1/(2*Df))
## Income      2.786182 1      1.669186
## Limit      234.028100 1      15.297977
## Rating      235.848259 1      15.357352
## Cards       1.448690 1      1.203615
## Age         1.051410 1      1.025383
## Education   1.019588 1      1.009747
## Gender      1.005849 1      1.002920
## Student     1.031517 1      1.015636
## Married     1.044638 1      1.022075
## Ethnicity   1.032231 2      1.007962
```

5.55

The matrix $\mathbf{X}$ considered in the estimation of the linear model

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

can be extracted from a linear model object with

```
X <- model.matrix(output)
head(X)
##      (Intercept) Income Limit Rating Cards Age Education GenderFemale
## 1             1  14.891  3606   283     2  34          11             0
## 2             1 106.025  6645   483     3  82          15             1
## 3             1 104.593  7075   514     4  71          11             0
## 4             1 148.924  9504   681     3  36          11             1
## 5             1  55.882  4897   357     2  68          16             0
## 6             1  80.180  8047   569     4  77          10             0
##      StudentYes MarriedYes EthnicityAsian EthnicityCaucasian
## 1             0           1              0                  1
## 2             1           1              1                  0
```

```
## 3      0      0      1      0
## 4      0      0      1      0
## 5      0      1      0      1
## 6      0      0      0      1
```

Note that X contains also the vector with unitary elements corresponding to the intercept.

5.56

X can be scaled, that is metric variables transformed as standardized, in order to work with standardized predictors, and obtain (semi-)standardized OLS coefficient estimates.

```
X.scaled <- X
X.scaled[, 2:7] <- scale(X[, 2:7])
apply(X.scaled, 2, sd)
##      (Intercept)      Income      Limit
##      0.0000000    1.0000000    1.0000000
##      Rating      Cards      Age
##      1.0000000    1.0000000    1.0000000
##      Education  GenderFemale  StudentYes
##      1.0000000    0.5003194    0.3003757
##      MarriedYes  EthnicityAsian EthnicityCaucasian
##      0.4877895    0.4364071    0.5006199
```

Using (semi-)standardized coefficient estimates is the simplest method to gain information about the relevance of single predictors. Sizes of coefficients are comparable within metric predictors and within dummy variables.

5.57

```
summary(lm(credit$Balance ~ 0 + X.scaled))
##
## Call:
## lm(formula = credit$Balance ~ 0 + X.scaled)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -161.64  -77.70  -13.49   53.98  318.20
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## X.scaled(Intercept)    478.867    12.507   38.288 < 2e-16 ***
## X.scaledIncome       -275.015     8.255  -33.314 < 2e-16 ***
## X.scaledLimit         440.651    75.660    5.824 1.21e-08 ***
## X.scaledRating        175.848    75.953    2.315  0.0211 *
## X.scaledCards          24.305     5.953    4.083 5.40e-05 ***
## X.scaledAge          -10.590     5.071   -2.088  0.0374 *
## X.scaledEducation     -3.434     4.994   -0.688  0.4921
## X.scaledGenderFemale  -10.653     9.914   -1.075  0.2832
## X.scaledStudentYes    425.747    16.723   25.459 < 2e-16 ***
## X.scaledMarriedYes     -8.534    10.363   -0.824  0.4107
## X.scaledEthnicityAsian  16.804    14.119    1.190  0.2347
## X.scaledEthnicityCaucasian 10.107    12.210    0.828  0.4083
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 98.79 on 388 degrees of freedom
## Multiple R-squared:  0.9803, Adjusted R-squared:  0.9797
## F-statistic: 1611 on 12 and 388 DF, p-value: < 2.2e-16
```

5.58

From now on the model with variables measured on their original scale will be considered. The object X contains the model matrix referred to this model.

We can find the number of variables in the complete model (comprehensive of dummy variables used for coding categorical variables).

```
(p <- ncol(X) - 1)
## [1] 11
```

5.59

The following code is reported to show how to build JWHT Fig. 6.1. You can skip it, and go to slide 5.67.

5.60

The package `sampling` contains the function `writesample` that extracts the $\binom{n}{k}$ k -ples combinations of size k from a set of n elements. We set the seed for reproducibility

```
set.seed(123)
library(sampling)
```

A list with p elements, whose elements are the variables selected in the k steps of best subset selection can be obtained with

```
a <- lapply(1:p, function(i) writesample(i, p))
```

5.61

We find indices of variables that are selected in the k steps of best subset selection (results is assigned as a list with p elements, rewriting the content of object `a`)

```
a <- lapply(1:(p - 1), function(i) {
  t(apply(a[[i]], 1, function(x) which(x == 1)))
})
```

The element corresponding to models with only 1 variable is to be redefined, as well as the element corresponding to the model with all p variables

```
a[[1]] <- matrix(1:p, p, 1)
a[[p]] <- matrix(1:p, 1, p)
```

5.62

We have to remember that final conclusion considers also the null model.

We assign to object `Y` the outcome and write a function to obtain RSS and Rsq for all models

```
Y <- credit$Balance
bestsubset0 <- function(k, j) {
  Xtmp <- X[, c(1, a[[k]][j, ] + 1)]
  betatmp <- solve(t(Xtmp) %*% Xtmp, t(Xtmp) %*% Y)
  Yhat <- Xtmp %*% betatmp
  c(RSStmp <- sum((Y - Yhat)^2), Rsqtmp <- cor(Y, Yhat)^2)
}
bestsubsetstats <- lapply(1:p, function(k) sapply(1:nrow(a[[k]]),
  function(j) bestsubset0(k, j)))
```

5.63

We plot the RSS, by considering also its value corresponding to the null model.

```
RSS <- (n - 1) * var(Y)
layout(matrix(1:2, 1, 2))
plot(0, RSS, xlim = c(0, 11), ylim = c(0, RSS), xlab = "Number of predictors",
  ylab = "Residual Sum of Squares")
graphtmp <- sapply(1:p, function(k) points(rep(k, ncol(bestsubsetstats[[k]])),
  bestsubsetstats[[k]][1, ]))
points(0:p, c(RSS, sapply(1:p, function(k) min(bestsubsetstats[[k]][1,
  ]))), pch = 16, col = "red")
lines(0:p, c(RSS, sapply(1:p, function(k) min(bestsubsetstats[[k]][1,
  ]))), col = "red")
```

5.64

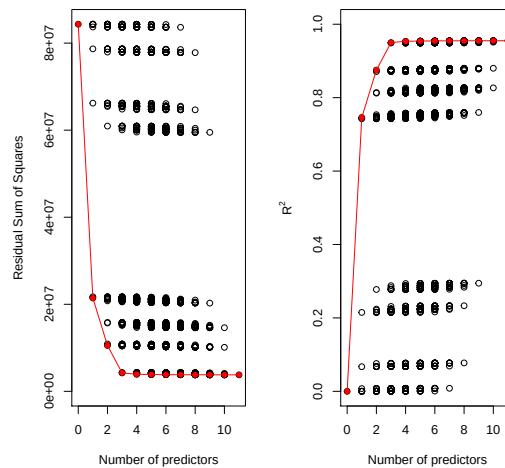
We plot the R^2 , by considering also its null value corresponding to the null model.

```

Rsq <- 0
plot(0, Rsq, xlim = c(0, 11), ylim = c(0, 1), xlab = "Number of predictors",
     ylab = expression(R^2))
graphtmp <- sapply(1:p, function(k) points(rep(k, ncol(bestsubsetstats[[k]])),
     bestsubsetstats[[k]][2, ]))
points(0:p, c(Rsq, sapply(1:p, function(k) max(bestsubsetstats[[k]][2,
  ]))), pch = 16, col = "red")
lines(0:p, c(Rsq, sapply(1:p, function(k) max(bestsubsetstats[[k]][2,
  ]))), col = "red")

```

5.65



5.66

R functions that perform subset selection are now considered.

The function `regsubsets` available in the package `leaps` returns the best models with $0:p$ predictors according to the R^2 (`r2`), adjusted R^2 (`adjr2`), Mallows's C_p (`Cp`) and BIC (`bic`) metrics.

The syntax of `regsubsets` resembles that of `lm` (the argument `nvmax` is set to 11 to consider models with all variables).

```

library(leaps)
regfit.full <- regsubsets(Balance ~ ., data = credit, nvmax = 11)

```

the sign "*" in the summary output flags variables that appear in a model.

5.67

```

(regfit.full.summary <- summary(regfit.full))
## Subset selection object
## Call: regsubsets.formula(Balance ~ ., data = credit, nvmax = 11)
## 11 Variables (and intercept)
##
##              Forced in Forced out
## Income                FALSE      FALSE
## Limit                  FALSE      FALSE
## Rating                 FALSE      FALSE
## Cards                  FALSE      FALSE
## Age                    FALSE      FALSE
## Education              FALSE      FALSE
## GenderFemale           FALSE      FALSE
## StudentYes             FALSE      FALSE
## MarriedYes             FALSE      FALSE
## EthnicityAsian         FALSE      FALSE
## EthnicityCaucasian     FALSE      FALSE
## 1 subsets of each size up to 11

```

```
## Selection Algorithm: exhaustive
##
## 1 ( 1 ) " " " " "*" " " " " " " " "
## 2 ( 1 ) "*" " " "*" " " " " " " " "
## 3 ( 1 ) "*" " " "*" " " " " " " " "
## 4 ( 1 ) "*" "*" " " "*" " " " " " "
## 5 ( 1 ) "*" "*" "*" "*" " " " " " "
## 6 ( 1 ) "*" "*" "*" "*" "*" " " " " "
## 7 ( 1 ) "*" "*" "*" "*" "*" " " " " "*"
## 8 ( 1 ) "*" "*" "*" "*" "*" " " " " "*"
## 9 ( 1 ) "*" "*" "*" "*" "*" " " " " "*"
## 10 ( 1 ) "*" "*" "*" "*" "*" "*" " " " "*"
## 11 ( 1 ) "*" "*" "*" "*" "*" "*" "*" " "*"
##
## StudentYes MarriedYes EthnicityAsian EthnicityCaucasian
## 1 ( 1 ) " " " " " " " "
## 2 ( 1 ) " " " " " " " "
## 3 ( 1 ) "*" " " " " " " " "
## 4 ( 1 ) "*" " " " " " " " "
## 5 ( 1 ) "*" " " " " " " " "
## 6 ( 1 ) "*" " " " " " " " "
## 7 ( 1 ) "*" " " " " " " " "
## 8 ( 1 ) "*" " " "*" " " " "
## 9 ( 1 ) "*" "*" "*" " " " "
## 10 ( 1 ) "*" "*" "*" "*" " " "
## 11 ( 1 ) "*" "*" "*" "*" "*" " "
```

5.68

```
names(regfit.full.summary)
## [1] "which" "rsq" "rss" "adjr2" "cp" "bic" "outmat"
## [8] "obj"
```

The rsq of the selected models with 1 : p variables can be extracted (the resulting vector length is p)

```
regfit.full.summary$rsq
## [1] 0.7458484 0.8751179 0.9498788 0.9535800 0.9541606
## [6] 0.9546879 0.9548167 0.9548880 0.9549636 0.9550468
## [11] 0.9551016
```

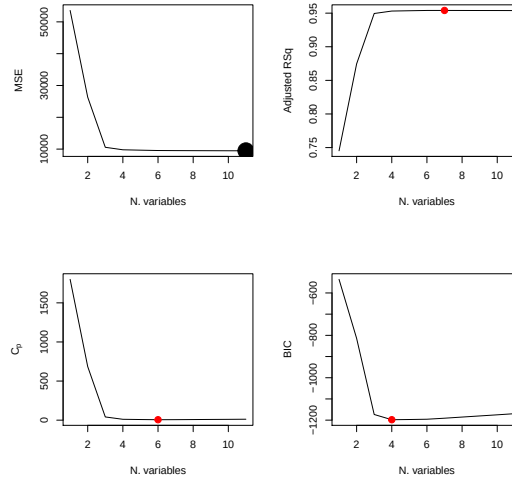
5.69

A multfigure plot with model selection results corresponding to different metrics is now produced.

```
layout(matrix(1:4, 2, 2, byrow = TRUE))
plot(regfit.full.summary$rss/n, xlab = "N. variables", ylab = "MSE",
     type = "l")
a <- which.min(regfit.full.summary$rss)
points(a, regfit.full.summary$rss[a]/n, cex = 5, pch = 20)
plot(regfit.full.summary$adjr2, xlab = "N. variables", ylab = "Adjusted RSq",
     type = "l")
a <- which.max(regfit.full.summary$adjr2)
points(a, regfit.full.summary$adjr2[a], col = "red", cex = 2,
       pch = 20)
plot(regfit.full.summary$cp, xlab = "N. variables", ylab = expression(C[p]),
     type = "l")
a <- which.min(regfit.full.summary$cp)
points(a, regfit.full.summary$cp[a], col = "red", cex = 2, pch = 20)
plot(regfit.full.summary$bic, xlab = "N. variables", ylab = "BIC",
     type = "l")
a <- which.min(regfit.full.summary$bic)
```

```
points(a, regfit.full.summary$bic[a], col = "red", cex = 2, pch = 20)
```

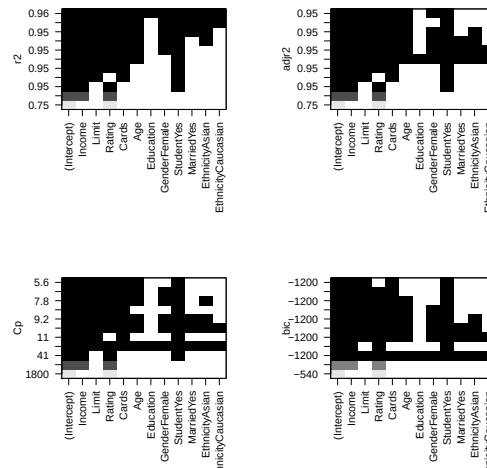
5.70



5.71

There exists also a plot method for `regsubsets` objects, that shows which variables are included in the best models chosen by best subset selection according to different metrics.

```
layout(matrix(1:4, 2, 2, byrow = TRUE))
graphtmp <- sapply(c("r2", "adjr2", "Cp", "bic"), function(i) plot(regfit.full,
  scale = i))
```



5.72

Best Subset model selection by using the validation set approach and the k -fold cross validation procedure is considered before analyzing the code to produce Forward and Backward Stepwise Selection.

As suggested by JWHT, cross validation can be performed as follows.

1. Preliminary selection has to be performed by considering the two steps:
 - (a) apply `regsubsets` to the training data set (training folds with k -fold CV);
 - (b) choose, by estimating the MSE_{test} on held out data (fold), the optimal number k^* of variables to include in the model;
 the so-called one-standard-error rule, see below, can be applied in order to find k^* .

2. Finally, in order to obtain more accurate parameter estimates, `regsubsets` is applied again on the whole data set, and coefficients of the model with k^* , as defined at step 2(b), variables are extracted.

5.73

Cross validation with the validation set is described first. JWHT suggest to consider a training data set with about 75% of the subjects.

```
quota.training <- 0.75
train <- sample(n, round(n * quota.training, 0))
regfit.best <- regsubsets(Balance ~ ., data = credit[train, ],
  nvmax = 11)
```

5.74

A predict method for the class `regsubsets` can be defined as

```
predict.regsubsets <- function(object, form = as.formula(Balance ~
  .), newdata, k) {
  test.matrix <- model.matrix(form, newdata)
  coef.k <- coef(object, k)
  espl.k <- names(coef.k)
  test.matrix[, espl.k] %*% coef.k
}
```

The line of code `form<-as.formula(object$call[[2]])` by JWHT was dropped, since the `regsubsets` object loses some properties when used within a function¹; it has been replaced by the argument

```
form = as.formula(Balance ~ .).
```

5.75

The following function `val.set` performs validation set, using the `predict.regsubsets` method.

```
val.set <- function(k) {
  train <- sample(n, round(n * quota.training, 0))
  regfit.best <- regsubsets(Balance ~ ., data = credit[train,
    ], nvmax = 11)
  newdata <- credit[-train, ]
  mean((credit$Balance[-train] - predict(object = regfit.best,
    form = Balance ~ ., newdata, k))^2)
}
```

5.76

We apply function `val.set` in order to obtain the optimal number k^* of variables to include in the final model.

```
(a <- sapply(1:p, function(k) val.set(k = k)))
## [1] 62978.38 27831.79 11094.61 11232.48 14674.54 11461.33
## [7] 10263.52 10846.98 12486.59 10403.26 10175.46
which.min(a)
## [1] 11
(fig6.3.vs <- sapply(1:p, function(k) sqrt(val.set(k = k))))
## [1] 218.14140 157.31881 100.52973 101.44497 92.80845
## [6] 104.59168 90.02868 103.91488 97.49690 97.70327
## [11] 102.34371
```

5.77

¹Maybe, this is the reason why there does not exist a prediction method for `regsubsets` objects. The authors of `regsubsets` state that the value of the function contains no user-serviceable parts. It is designed to be processed by `summary.regsubsets`.

Finally, `regsubsets` has to be replicated on the whole data set with the final settings obtained with the k -fold Cross Validation procedure, in order to obtain more accurate parameter estimates.

```
regfit.best <- regsubsets(Balance ~ ., data = credit, nvmax = 11)
coef(regfit.best, which.min(a))
```

##	(Intercept)	Income	Limit
##	-479.2078706	-7.8031018	0.1909067
##	Rating	Cards	Age
##	1.1365265	17.7244836	-0.6139088
##	Education	GenderFemale	StudentYes
##	-1.0988553	-10.6532477	425.7473595
##	MarriedYes	EthnicityAsian	EthnicityCaucasian
##	-8.5339006	16.8041792	10.1070252

5.78

One must always be aware of the possible variability of results that characterizes the validation set procedure

```
table(replicate(1000, which.min(sapply(1:p, function(k) val.set(k = k))))))
```

##										
##	3	4	5	6	7	8	9	10	11	
##	18	115	123	123	152	110	121	126	112	

5.79

A k -fold Cross Validation ($k = 10$) is now considered.

```
k <- 10
set.seed(123)
folds <- sample(1:k, n, replace = TRUE)
```

In case n is a multiple of k , the following code gives folds of equal size.

```
folds <- rep(1:k, each = n/k) [sample(n)]
```

5.80

Matrix A is populated by setting element $a[i, j]$ equal to the partial MSE_{test} estimate referred to the i th fold for the model with j variables.

```
A <- matrix(NA, k, p)
for (j in 1:k) {
  best.fit <- regsubsets(Balance ~ ., data = credit[folds !=
    j, ], nvmax = 11)
  for (i in 1:p) A[j, i] <- mean((credit$Balance[folds == j] -
    predict(object = best.fit, newdata = credit[folds ==
    j, ], k = i))^2)
}
```

Estimates of MSE_{test} for models with $1:p$ variables can be obtained by averaging partial estimates.

```
mean.cv.errors <- apply(A, 2, mean)
which.min(mean.cv.errors)
```

##	[1] 6
----	-------

5.81

`regsubsets` has to be replicated on the whole data set with the final settings, achieved with k -fold CV, in order to obtain more accurate parameter estimates

```
regfit.best <- regsubsets(Balance ~ ., data = credit, nvmax = 11)
coef(regfit.best, which.min(mean.cv.errors))
```

##	(Intercept)	Income	Limit	Rating
##	-493.7341870	-7.7950824	0.1936914	1.0911874
##	Cards	Age	StudentYes	

```
##      18.2118976      -0.6240560      425.6099369
```

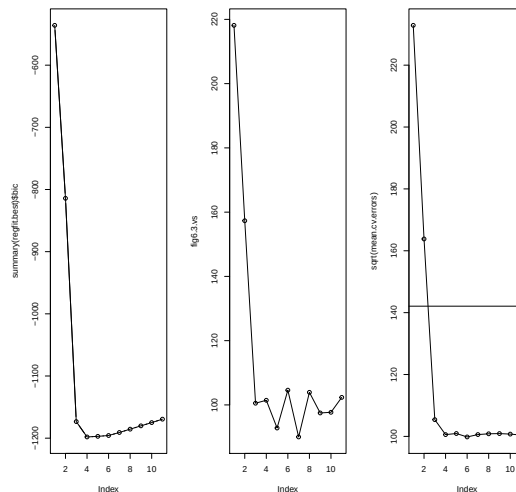
5.82

Observe that also k -fold cross validation results can depend on how the k folds are created. So in order to choose the best number of variables k^* , one can have recourse to the so-called one-standard-error rule.

First calculate the standard error of the estimated test MSE for each model size, and then select the smallest model for which the estimated test error is within one standard error of the lowest point on the curve

```
layout(matrix(1:3, 1, 3))
plot(summary(regfit.best)$bic, type = "l")
points(1:p, summary(regfit.best)$bic, type = "b") # bic scale != from JWHT
plot(fig6.3.vs, type = "l")
points(1:p, fig6.3.vs)
plot(sqrt(mean.cv.errors), type = "l")
points(1:p, sqrt(mean.cv.errors))
sd.cv.errors <- sd(sqrt(mean.cv.errors))
abline(h = min(sqrt(mean.cv.errors)) + sd.cv.errors)
```

5.83



5.84

In this case the best model is the one containing 3 variables. We thus extract its coefficients from the `regfit.best` object referred to the model estimated on whole data

```
coef(regfit.best, 3)
## (Intercept)      Income      Rating  StudentYes
## -581.078888    -7.874931    3.987472   418.760284
```

5.85

Forward and Backward Stepwise selection can be performed with `regsubsets` by setting the argument `method="forward"` and `method="backward"`, respectively.

5.86

```
regfit.fwd <- regsubsets(Balance ~ ., data = credit, nvmax = 11, method = "forward")
summary(regfit.fwd)
## Subset selection object
## Call: regsubsets.formula(Balance ~ ., data = credit, nvmax = 11, method = "forward")
## 11 Variables (and intercept)
##
##              Forced in Forced out
## Income              FALSE      FALSE
## Limit               FALSE      FALSE
## Rating              FALSE      FALSE
## Cards               FALSE      FALSE
```

```
## Age FALSE FALSE
## Education FALSE FALSE
## GenderFemale FALSE FALSE
## StudentYes FALSE FALSE
## MarriedYes FALSE FALSE
## EthnicityAsian FALSE FALSE
## EthnicityCaucasian FALSE FALSE
## 1 subsets of each size up to 11
## Selection Algorithm: forward
##      Income Limit Rating Cards Age Education GenderFemale
## 1 ( 1 ) " " " " "*" " " " " " " " "
## 2 ( 1 ) "*" " " "*" " " " " " " " "
## 3 ( 1 ) "*" " " "*" " " " " " " " "
## 4 ( 1 ) "*" "*" "*" " " " " " " " "
## 5 ( 1 ) "*" "*" "*" "*" " " " " " "
## 6 ( 1 ) "*" "*" "*" "*" "*" " " " " "
## 7 ( 1 ) "*" "*" "*" "*" "*" "*" " " "*"
## 8 ( 1 ) "*" "*" "*" "*" "*" "*" " " "*"
## 9 ( 1 ) "*" "*" "*" "*" "*" "*" " " "*"
## 10 ( 1 ) "*" "*" "*" "*" "*" "*" " " "*"
## 11 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*"
##      StudentYes MarriedYes EthnicityAsian EthnicityCaucasian
## 1 ( 1 ) " " " " " " " "
## 2 ( 1 ) " " " " " " " "
## 3 ( 1 ) "*" " " " " " " " "
## 4 ( 1 ) "*" " " " " " " " "
## 5 ( 1 ) "*" " " " " " " " "
## 6 ( 1 ) "*" " " " " " " " "
## 7 ( 1 ) "*" " " " " " " " "
## 8 ( 1 ) "*" " " "*" " " " "
## 9 ( 1 ) "*" "*" "*" " " " "
## 10 ( 1 ) "*" "*" "*" "*" " "
## 11 ( 1 ) "*" "*" "*" "*" "*" "
```

5.87

```
regfit.bwd <- regsubsets(Balance ~ ., data = credit, nvmax = 11, method = "backward")
summary(regfit.bwd)
## Subset selection object
## Call: regsubsets.formula(Balance ~ ., data = credit, nvmax = 11, method = "backward")
## 11 Variables (and intercept)
##      Forced in Forced out
## Income FALSE FALSE
## Limit FALSE FALSE
## Rating FALSE FALSE
## Cards FALSE FALSE
## Age FALSE FALSE
## Education FALSE FALSE
## GenderFemale FALSE FALSE
## StudentYes FALSE FALSE
## MarriedYes FALSE FALSE
## EthnicityAsian FALSE FALSE
## EthnicityCaucasian FALSE FALSE
## 1 subsets of each size up to 11
## Selection Algorithm: backward
##      Income Limit Rating Cards Age Education GenderFemale
## 1 ( 1 ) " " "*" " " " " " " " "
## 2 ( 1 ) "*" "*" " " " " " " " "
```

```
## 3 ( 1 ) "*" "*" " " " " " " " " " "
## 4 ( 1 ) "*" "*" " " "*" " " " " " "
## 5 ( 1 ) "*" "*" "*" "*" " " " " " "
## 6 ( 1 ) "*" "*" "*" "*" "*" " " " "
## 7 ( 1 ) "*" "*" "*" "*" "*" " " " "*"
## 8 ( 1 ) "*" "*" "*" "*" "*" " " " "*"
## 9 ( 1 ) "*" "*" "*" "*" "*" " " " "*"
## 10 ( 1 ) "*" "*" "*" "*" "*" " " " "*"
## 11 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*"
##
## StudentYes MarriedYes EthnicityAsian EthnicityCaucasian
## 1 ( 1 ) " " " " " " " "
## 2 ( 1 ) " " " " " " " "
## 3 ( 1 ) "*" " " " " " "
## 4 ( 1 ) "*" " " " " " "
## 5 ( 1 ) "*" " " " " " "
## 6 ( 1 ) "*" " " " " " "
## 7 ( 1 ) "*" " " " " " "
## 8 ( 1 ) "*" " " "*" " "
## 9 ( 1 ) "*" "*" "*" " "
## 10 ( 1 ) "*" "*" "*" "*"
## 11 ( 1 ) "*" "*" "*" "*"

```

5.88

We do not perform the cross validation selection procedure with regard Forward and Backward stepwise procedures; we only observe that the three methods return practically the same model (with 3 variables included). The only difference is that the Backward procedure inserts `Limit` instead of `Rating`, but these two variables have practically the same information content.

```
coef(regfit.full, 3)
## (Intercept)      Income      Rating  StudentYes
## -581.078888   -7.874931   3.987472  418.760284
coef(regfit.fwd, 3)
## (Intercept)      Income      Rating  StudentYes
## -581.078888   -7.874931   3.987472  418.760284
coef(regfit.bwd, 3)
## (Intercept)      Income      Limit    StudentYes
## -432.3374179   -7.9016203   0.2675379  427.0232667

```

5.89

A possible solution to the multicollinearity issue, due to the presence of the highly correlated `Limit` and `Rating` variables, is to analyze data by defining a new variable `Average.Worthiness` that resumes `Limit` and `Rating`. `Average.Worthiness` is obtained as average of the two standardized variables.

```
credit <- transform(credit, Average.Worthiness = ((Limit -
  mean(Limit))/sd(Limit) + (Rating - mean(Rating))/sd(Rating))/2)

```

5.90

`Limit` and `Rating` are dropped and a new regression model estimated.

```
names(credit)
## [1] "Income"      "Limit"
## [3] "Rating"      "Cards"
## [5] "Age"         "Education"
## [7] "Gender"      "Student"
## [9] "Married"     "Ethnicity"
## [11] "Balance"     "Average.Worthiness"
credit <- credit[, -(2:3)]
output <- lm(Balance ~ ., data = credit)

```

5.91

```
summary(output)

##
## Call:
## lm(formula = Balance ~ ., data = credit)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -172.66  -78.97  -11.30   59.48  312.06
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    839.4656     33.1996  25.285 < 2e-16 ***
## Income         -7.8121      0.2348 -33.272 < 2e-16 ***
## Cards          13.5591      3.6391   3.726 0.000223 ***
## Age            -0.6203      0.2947  -2.105 0.035958 *
## Education      -0.8826      1.5974  -0.553 0.580902
## GenderFemale  -10.7023      9.9402  -1.077 0.282294
## StudentYes    422.9972     16.6925  25.341 < 2e-16 ***
## MarriedYes    -10.5166     10.3279  -1.018 0.309184
## EthnicityAsian  18.3364     14.1291   1.298 0.195135
## EthnicityCaucasian 10.2707     12.2418   0.839 0.401994
## Average.Worthiness 617.0132      8.1901  75.336 < 2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 99.05 on 389 degrees of freedom
## Multiple R-squared:  0.9547, Adjusted R-squared:  0.9536
## F-statistic: 820.7 on 10 and 389 DF, p-value: < 2.2e-16

vif(output)

##              GVIF Df GVIF^(1/(2*Df))
## Income         2.784849  1      1.668787
## Cards          1.012711  1      1.006335
## Age            1.051245  1      1.025302
## Education      1.013484  1      1.006720
## Gender         1.005841  1      1.002916
## Student        1.022398  1      1.011137
## Married        1.032139  1      1.015942
## Ethnicity      1.026976  2      1.006677
## Average.Worthiness 2.723656  1      1.650350
```

5.92

5.93